C++ 基础入门

典型题精编与详细解析

10 道单选 • 5 道多选 • 5 道填空 • 4 道综合题

适用范围

第一个 C++ 程序、源文件包含、`main`、标准输出、命名空间、注释、变量声明与初始化、块作用域、整数运算及基础格式化输出

编制日期：2026 年 7 月 25 日

基于公开权威技术资料原创改编；完整来源与映射见文末

目录

1 原图内容数字化重排 **2**

1.1 忠实转写版 2

1.2 规范校订版 3

2 试卷说明与难度结构 **4**

3 题目 **5**

3.1 一、单项选择题（每题 2 分，共 20 分） 5

3.2 二、多项选择题（每题 4 分，共 20 分） 8

3.3 三、填空题（每题 3 分，共 15 分） 9

3.4 四、综合题（共 45 分） 11

4 参考答案与极详解析 **13**

4.1 答案速查 13

4.2 一、单项选择题详解 13

4.3 二、多项选择题详解 16

4.4 三、填空题详解 18

4.5 四、综合题详解 19

5 来源、检索说明与题目映射 **26**

5.1 权威资料 26

5.2 题目–来源映射 27

5.3 最终复核清单 27

1 原图内容数字化重排

转写原则

本节先按照片中**可辨认的手写内容**忠实转写，再给出能够按标准 C++17 编译的规范校订版。照片下方存在轻微裁切；无法从图中确认的内容不擅自补写。代码关键字大小写、缺失的命名空间限定以及 `system("pause")` 的可移植性问题，统一在“校订说明”中处理。

1.1 忠实转写版

第一章：C++ 入门

§1 Hello, world 代码编写：

```cpp
#include <iostream>
using namespace std;

int main()
{
    cout << "hello, world" << endl;
    system("pause");
    return 0;
}
```

§2 注释

1. 作用：在代码中加一些说明和注释，方便阅读代码。
2. 格式：
   ① 单行注释：`//`
   ② 多行注释：`/* ... */`

§3 变量

1. 意义：方便管理内存空间。
2. 创建语法：数据类型 变量名 = 变量初始值;
3. 例：

```cpp
#include <iostream>

int main()
{
    int a = 10;
    cout << "a = " << a << endl;
    system("pause");
    return 0;
}
```

图像辨识说明

第二段示例在照片可见区域内没有出现 `using namespace std;`，但使用了未限定的 `cout` 与 `endl`；因此忠实转写后并不保证编译通过。首段手写的 `using` 大小写略含混，本册按 C++ 正确关键字的小写形式录入。

1.2 规范校订版

Hello, world

```cpp
#include <iostream>

int main()
{
    std::cout << "hello, world" << '\n';
    return 0;
}
```

变量示例

```cpp
#include <iostream>

int main()
{
    int a{10};
    std::cout << "a = " << a << '\n';
    return 0;
}
```

原写法	规范处理	原因
`Using`（若按大写辨认）	`using`	C++ 关键字区分大小写；大写形式不是该关键字。
`cout`、`endl`	`std::cout`、`std::endl`	标准流对象及操纵符位于 `std` 命名空间。显式限定可减少名称冲突。
`endl`	普通换行优先写`'\n'`	`std::endl` 不只换行，还会刷新输出缓冲区；不需要立即刷新时，普通换行更直接。
`system("pause")`	教学示例中删除	`system` 会把字符串交给宿主命令解释器；`pause` 是环境相关命令，不是标准 C++ 的“暂停语句”，且原代码没有显式包含声明它所需的头文件。
`int a = 10;`	`int a{10};`	二者在这里结果相同；花括号初始化还能拒绝某些意外的窄化转换。

本册统一约定

除专门考查错误代码外，所有程序均以**托管环境下的标准 C++17** 为基线；标准库名称显式使用 `std::`；普通换行优先使用`'\n'`；题目不依赖某个 IDE、操作系统或编译器扩展。

2 试卷说明与难度结构

题型	数量	分值	作答规则
单项选择题	10	$10 \times 2 = 20$	每题只有一个最符合题意的选项。
多项选择题	5	$5 \times 4 = 20$	每题至少两个正确选项；多选、错选均不得分。
填空题	5	$5 \times 3 = 15$	代码标记与大小写必须符合 C++ 语法。
综合题	4	$10 + 10 + 12 + 13 = 45$	写出过程；程序题以标准 C++17 为准。
合计	24	100	建议用时 90–120 分钟。

难度层级	题量	设计目标
基础	9	检查语法骨架、术语和直接读码能力。
中等	12	检查多知识点组合、作用域与数据流追踪。
提高	3	检查语言细节、可移植性与精度意识。

先做题，后看解析

为满足“答案在最后呈现”的要求，第 3 节只给题目；全部答案、逐项辨析、推导过程、参考程序与边界测试从第 4 节开始集中给出。

3 题目

3.1 一、单项选择题（每题 2 分，共 20 分）

单选 1 [基础] （2 分）

下面哪段代码是标准 C++17 中能够输出一行 `Hello, C++!` 的完整程序？

A

```
#include <iostream>
int main() {
    std::cout << "Hello, C++!" << '\n';
    return 0;
}
```

```
#include <iostream>
void main() {
    std::cout << "Hello, C++!" << '\n';
}
```

C

```
#include <iostream>
Using namespace std;
int main() {
    cout << "Hello, C++!" << '\n';
}
```

```
#include <iostream>
int main() {
    cout << "Hello, C++!" << '\n';
}
```

知识点：main、标准输出、命名空间、关键字大小写

单选 2 [基础] （2 分）

`#include <iostream>` 在本册示例中的主要作用是：

A. 在运行时创建一个名为 `cout` 的局部变量

B. 使翻译单元获得标准流相关声明，从而能够使用 `std::cout` 等对象

C. 自动执行 `main` 函数

D. 自动把 `std` 命名空间中的所有名称引入全局作用域

知识点：源文件包含、头文件、声明

单选 3 [基础] （2 分）

执行下列语句后，标准输出的可见内容是什么？

```
std::cout << "sum=" << 2 + 3 << '\n';
```

A. `sum=2+3`

B. `sum=23`

C. `sum=5`，随后换行

D. 程序必然编译错误

知识点：流插入运算、表达式求值、输出次序

单选 4 [基础] （2 分）

对一个正常状态的输出流 `os`，`std::endl(os)` 的核心效果是：

A. 只插入一个空格

B. 插入换行字符，然后刷新输出流

C. 只刷新输出流，不插入任何字符

D. 终止整个程序

知识点：`std::endl`、换行与刷新

单选 5 [中等] （2 分）

观察下面的片段：

```
/* outer /* inner */ std::cout << "X"; */
```

下列判断正确的是：

A. 外层和内层注释会正确嵌套，整行都被忽略

B. 第一个 `*/` 已结束注释，后面的记号会作为代码继续处理，片段因此不合法

C. 两个 `/*` 会自动合并成单行注释

D. 只有字符串`"X"` 会被当作注释

知识点：块注释不嵌套

单选 6 [基础] （2 分）

下列名称中，哪一个可作为普通 C++ 变量名？

A. `2score`

B. `double`

C. `total-score`

D. `score_2`

知识点：标识符与关键字

单选 7 [中等] （2 分）

下面程序片段输出什么？

```
int a{2};
```

```
int b{a + 3};
a = 10;
std::cout << b << '\n';
```

A. 2

B. 5

C. 10

D. 13

知识点：初始化、赋值、变量值的独立性

单选 8 [中等] （2 分）

下面程序片段的连续输出是：

```
int x{1};
{
    int x{2};
    std::cout << x;
}
std::cout << x;
```

A. 11

B. 12

C. 21

D. 22

知识点：块作用域、同名变量遮蔽

单选 9 [中等] （2 分）

若 `int a{7}; int b{2};`，表达式 `a / b` 的值是：

A. 3

B. 3.5

C. 4

D. 结果未定义

知识点：整数除法

单选 10 [提高] （2 分）

下列声明中，哪一项会因为列表初始化中的**窄化转换**而不合法？

A. `int a = 3.9;`

B. `double b{3};`

C. `int c{3};`

D. `int d{3.9};`

知识点：列表初始化、窄化转换

3.2 二、多项选择题（每题 4 分，共 20 分）

多选 1 [中等] （4 分）

关于托管环境中的 `main` 函数，下列说法正确的有：

A. 一个程序应恰有一个属于全局作用域的 `main` 函数

B. `main` 的声明返回类型应为 `int`

C. 实现应允许 `int main()` 和 `int main(int, char**)` 这两种形式

D. 控制流到达 `main` 函数体末尾，效果等价于返回 0

E. `void main()` 是标准要求实现必须接受的形式

知识点：`main` 的标准形式与返回

多选 2 [中等] （4 分）

关于 C++ 注释，下列说法正确的有：

A. `//` 注释在下一个换行字符之前结束

B. `/* ... */` 注释可以跨越多行

C. `/* ... */` 注释可以任意层数嵌套

D. 在 `//` 注释内部，`/*` 与 `*/` 没有特殊的注释控制含义

E. 注释会在程序运行时逐条执行

知识点：两类注释及其词法规则

多选 3 [中等] （4 分）

下列声明中，哪些会定义一个**类型为 `int` 且初值为 0 的变量**？

A. `int a = 0;`

B. `int b(0);`

C. `int c{0};`

D. `auto d = 0;`

E. `int e();`

知识点：复制初始化、直接初始化、列表初始化、`auto`、函数声明

多选 4 [中等] （4 分）

已知源文件顶部有 `#include <iostream>`，下列哪些片段放在 `main` 函数体中可以合法输出字符 `A`？

A. `std::cout << "A";`

B. `using namespace std; cout << "A";`

C. `using std::cout; cout << "A";`

D. `cout << "A";`（此前没有任何 `using` 声明或指令）

E. `std::cout << "A" << std::endl;`

知识点：限定名、using 指令、using 声明、标准输出

多选 5 [提高] （4 分）

关于入门代码的正确性、可移植性和输出效率，下列说法正确的有：

A. `system("pause")` 会把命令交给宿主命令解释器；它不是标准 C++ 定义的通用暂停方式

B. `std::endl` 会插入换行并刷新输出流

C. 只需要换行而不要求立即刷新时，`'\n'` 通常足够

D. 未初始化的自动存储期局部 `int` 在 C++17 中可以安全地假定为 0

E. 花括号列表初始化能够拒绝某些窄化转换

知识点：宿主命令、流刷新、未初始化值、列表初始化

3.3 三、填空题（每题 3 分，共 15 分）

填空 1 [基础] （3 分）

补全一个标准 C++17 的输出程序：

```
#include <_____>

int _____()
{
    std::_____ << "Hello, C++!" << '\n';
    return 0;
}
```

知识点：头文件、入口函数、标准输出对象

填空 2 [基础] （3 分）

C++ 单行注释的起始记号是_____；块注释的起始记号是_____，结束记号是_____。

知识点：注释记号

填空 3 [基础] （3 分）

在声明 `int score{95};` 中，数据类型是_____，变量名是_____，初始值是_____。

知识点：变量声明的组成

填空 4 [中等] （3 分）

写出下面片段的精确输出（两个整数之间有一个空格）：

```
int total{17};
int group{5};
std::cout << total / group << ' ' << total % group;
```

输出：_____

知识点：整数商与余数

填空 5 [中等] （3 分）

`std::endl` 先向输出流插入一个_____ 字符，然后调用流的_____ 操作。

知识点：`std::endl` 的精确语义

3.4 四、综合题（共 45 分）

综合 1：把手写程序修到可移植 [基础] （10 分）

下面代码模拟了原图中常见的手写疏漏：

```
#Include <iostream>
Using namespace std

int Main()
{
    int a = 10
    cout << "a = " << a << endl
    system("pause");
    return 0;
}
```

1. 找出并解释至少 7 处语法、名称查找或可移植性问题。（6 分）
2. 写出一个简洁、可移植、能输出 `a = 10` 并换行的标准 C++17 程序。（4 分）

知识点：预处理指令、大小写、分号、main、命名空间、标准输出、可移植性

综合 2：变量状态与块作用域追踪 [中等] （10 分）

阅读程序，不运行编译器，回答问题：

```
#include <iostream>

int main()
{
    int a{7};
    int b{2};
    int q{a / b};
    int r{a % b};

    std::cout << q << ' ' << r << '\n';
    {
        int a{q + r};
        b = a * 2;
        std::cout << a << ' ' << b << '\n';
    }
    std::cout << a << ' ' << b << '\n';
    return 0;
}
```

1. 写出三行精确输出。（4 分）
2. 说明内层 `a` 与外层 `a` 的关系，以及离开内层块后 `a`、`b` 各为何值。（4 分）
3. 把内层变量改名为 `innerA`，在不改变输出的前提下改写内层块。（2 分）

知识点：整数除法与余数、初始化、赋值、遮蔽、对象生存期

综合 3：总秒数拆分 [中等] （12 分）

题目。输入一个非负整数 S，表示总秒数。将其拆成“天、小时、分钟、秒”，输出四个以空格分隔的整数 d, h, m, s。其中

$$0 \le h < 24, \qquad 0 \le m < 60, \qquad 0 \le s < 60.$$

输入范围： $0 \le S \le 10^{12}$。

样例输入： `90061`

样例输出： `1 1 1 1`

1. 设计变量与计算步骤，说明为什么要选择合适的数据类型。（4 分）
2. 写出标准 C++17 程序。（6 分）
3. 给出时间复杂度与额外空间复杂度。（2 分）

知识点：标准输入输出、`long long`、整数除法、取余、逐级分解

综合 4：课程总评与格式化输出 [提高] （13 分）

题目。某课程总评由平时成绩、期中成绩、期末成绩按 20%、30%、50% 加权得到。输入三个 $[0, 100]$ 范围内的整数 p, m, f，按

$$\text{finalScore} = 0.20p + 0.30m + 0.50f$$

计算总评，并固定输出两位小数。

样例输入： `85 90 92`

样例输出： `90.00`

1. 写出标准 C++17 程序。（7 分）
2. 解释为什么不能把权重直接写成 `20 / 100`、`30 / 100`、`50 / 100` 后再与成绩相乘。（3 分）
3. 说明 `std::fixed` 与 `std::setprecision(2)` 的配合作用，并给出复杂度。（3 分）

知识点：浮点变量、整数除法陷阱、表达式建模、`<iomanip>`、格式化输出

4 参考答案与极详解析

4.1 答案速查

客观题答案表

单选 1.A；2.B；3.C；4.B；5.B；6.D；7.B；8.C；9.A；10.D。

多选 1.ABCD；2.ABD；3.ABCD；4.ABCE；5.ABCE。

填空 1. `iostream`、`main`、`cout`；2. `//`、`/*`、`*/`；3. `int`、`score`、`95`；4. `3 2`；5. 换行（`'\n'`）、`flush`。

怎样使用解析

不要只核对字母。每道选择题的错误选项都对应一种高频误区；能够解释“为什么错”，才说明真正掌握。涉及程序输出时，建议先画变量状态表，再与本册推导对照。

4.2 一、单项选择题详解

单选 1：答案 A

结论：A 是标准、完整且可移植的写法。

- **A 正确。** `<iostream>` 提供标准流相关声明；`std::cout` 使用了正确的命名空间限定；`main` 的返回类型为 `int`；`return 0;` 表示正常结束。字符字面量`'\n'` 输出一个换行字符。
- **B 错误。** 托管环境下的标准 C++ 要求 `main` 的声明返回类型为 `int`。`void main()` 可能被个别实现当作扩展接受，但不是标准答案，也不具备可移植性。
- **C 错误。** C++ 区分大小写，关键字是全小写 `using`，不是 `Using`。本选项因此在词法/语法分析阶段失败。顺带说明：若把它改成小写，`main` 末尾省略显式 `return 0;` 本身并不构成错误；到达 `main` 末尾等价于返回 0。
- **D 错误。** `cout` 位于 `std` 命名空间。单纯包含 `<iostream>` 只让相应声明可用，并不会把 `std` 中的所有名字自动变成不限定名；因此此处找不到裸写的 `cout`。

依据：S1、S3、S6

单选 2：答案 B

结论：`#include <iostream>` 使当前翻译单元获得标准流相关声明。

- **B 正确。** 源文件包含在预处理阶段发生；对标准头 `<iostream>` 的包含，使程序可以按该头提供的接口使用 `std::cin`、`std::cout`、`std::cerr` 等标准流对象。
- **A 错误。** `std::cout` 不是运行到该行时才创建的局部变量，更不是名为 `cout` 的用户变量。它是标准库声明的标准输出流对象。
- **C 错误。** `main` 的调用属于程序启动语义；`#include` 不负责执行函数。
- **D 错误。** 包含头文件与 using 指令是两种不同机制。前者使声明进入翻译单元，后者影响不限定名称查找；包含 `<iostream>` 不等于 `using namespace std;`。

易错辨析： 尖括号形式通常用于实现提供的标准/系统头；双引号形式通常先从当前源文件附近

搜索项目自有头。具体搜索位置由实现规定，但二者都不是“运行时导入”。依据：S3、S5、S11

单选 3：答案 C

表达式可按下面的逻辑分组理解：

$$((\texttt{std::cout} \ll \texttt{"sum="}) \ll (2+3)) \ll \texttt{'\textbackslash n'}.$$

先计算整数加法 $2+3=5$，然后流插入从左到右依次输出字符串 `sum=`、整数 5 和换行字符。所以可见内容为 `sum=5`，随后换行，答案为 C。

- A 把源代码文本 `2+3` 错当成字符串；只有写在引号中的字符才会照字面输出。
- B 把两个整数误当成两个独立的流插入项；实际代码中它们由加号组成一个表达式。
- D 错在忽视了 `<iostream>` 与正确的 `std::cout` 写法。

依据：S3

单选 4：答案 B

标准所描述的效果是：先调用输出流的字符写入操作插入换行字符，随后调用 `flush()` 刷新流。因此 B 完整，A、C 都只描述了部分或错误效果，D 则把流操纵符误解成程序终止函数。
实践含义：刷新会推动缓冲数据尽快交给下层设备，交互式提示、日志关键点等场景可能需要；普通批量输出若每行都无条件刷新，可能造成不必要的开销。因此只要求换行时，常写`'\n'`。依据：S4

单选 5：答案 B

C++ 的 `/* ... */` 注释**不能嵌套**。词法分析从第一个 `/*` 开始，在随后遇到的**第一个** `*/` 处结束注释。因此原片段等价于留下近似如下的待分析代码：

```
std::cout << "X"; */
```

前面的输出语句会被当成代码，末尾孤立的 `*/` 不能组成合法记号序列，片段不合法。
A 错在假定块注释可嵌套；C 混淆块注释与行注释；D 错在认为字符串内容决定注释边界。**稳妥做法**：临时屏蔽一大段可能自身含块注释的代码时，优先使用编辑器批量加 `//`，或使用条件编译，而不要再套一层块注释。依据：S2

单选 6：答案 D

- **D 正确**：`score_2` 以字母开头，后续字符均为字母、数字或下划线，且它不是关键字。
- **A 错误**：`2score` 以数字开头，不能作为普通标识符。
- **B 错误**：`double` 是内置浮点类型关键字，关键字不能重新用作变量名。
- **C 错误**：连字符会被识别成减号，`total-score` 会被解析为减法表达式，不是一个标识符。

实际工程还应避开为实现或标准库保留的下划线命名模式；入门阶段采用有意义的小写单词及下划线组合最稳妥。依据：S12

单选 7：答案 B

逐句追踪如下：

时刻	a	b	说明
执行第 1 句后	2	–	a 初始化为 2
执行第 2 句后	2	5	用当时的 $a+3=5$ 初始化 b
执行第 3 句后	10	5	赋值只修改 a
输出时	10	5	输出 b

b 保存的是初始化表达式计算所得的整数值 5，它不会与 a 建立“自动同步”关系。因此输出 5，答案 B。A 忘记了加 3；C 错把 b 当成 a 的别名；D 错在用 a 的新值重新计算了一个并未再次执行的初始化式。依据：S7、S12

单选 8：答案 C

外层 x 初始化为 1。进入内层复合语句后，新声明的内层 x 初始化为 2，并在该块内遮蔽外层同名变量，所以第一次输出 2。离开内层块后，内层对象的生存期结束，外层 x 再次可见且仍为 1，所以第二次输出 1。两次输出之间没有空格或换行，最终连续显示 21，答案 C。

“同名”不表示“同一个对象”：两次声明分别创建两个变量，具有不同作用域和存储。依据：S8

单选 9：答案 A

两个操作数都是 int，所以进行整数除法。$7=2\times3+1$，商为 3、余数为 1；a / b 只得到商 3。标准表述是舍弃代数商的小数部分（向 0 截断），因此答案 A。

B 把整数除法误当成浮点除法；C 做了四舍五入，而整数除法不这样工作；D 也不对，因为除数 b 为 2 而非 0，且结果可表示。若要得到 3.5，至少让一个操作数参与浮点运算，例如 static_cast<double>(a) / b。依据：S9

单选 10：答案 D

- **D 不合法。** 3.9 是浮点值，把它通过花括号列表初始化为 int 需要从浮点到整数的窄化转换；列表初始化在顶层拒绝这种转换。
- **A 合法但可能产生警告。**复制初始化允许把 3.9 转成 int，结果为 3（小数部分被舍弃）。“能编译”不代表“意图一定正确”。
- **B 合法。**整数常量 3 可表示为 double，这里不构成被禁止的窄化。
- **C 合法。**类型和值均为整数，且 3 可由 int 表示。

这正是入门代码推荐 int a{10}; 的一个理由：花括号能让某些可能丢失信息的初始化尽早暴露为编译错误。依据：S7

4.3 二、多项选择题详解

多选 1：答案 A、B、C、D

- **A 正确**。托管程序应包含恰好一个属于全局作用域的 `main`。
- **B 正确**。 `main` 的声明返回类型应为 `int`。
- **C 正确**。实现应允许无参数形式，以及由参数个数和参数字符串指针构成的形式；`char**` 与形参位置的 `char* argv[]` 表达的是相应参数形式。
- **D 正确**。控制流到达 `main` 复合语句末尾，效果等价于返回 0。
- **E 错误**。 `void main()` 不是标准要求必须接受的形式。

注意：“到达末尾可省略 `return 0;`”是 `main` 的特殊规则，不能随意推广到所有返回非 `void` 的普通函数。依据：S1

多选 2：答案 A、B、D

- **A 正确**：`//` 注释在下一个换行字符之前终止。
- **B 正确**：`/* ... */` 以结束记号为界，可以自然跨越多行。
- **C 错误**：块注释不嵌套；第一个结束记号就会终止当前块注释。
- **D 正确**：在行注释内部，`/*` 与 `*/` 只是普通字符，不会改变该行注释的终点。
- **E 错误**：注释在翻译早期被当作词法间隔处理，不是运行期语句。

完整答案必须是 A、B、D；多选题少选 D 仍不完整。依据：S2

多选 3：答案 A、B、C、D

- A 是复制初始化，定义 `int` 变量 `a`，初值 0。
- B 是圆括号形式的直接初始化，定义 `int` 变量 `b`，初值 0。
- C 是直接列表初始化，定义 `int` 变量 `c`，初值 0。
- D 由整数字面量 `0` 推导 `auto` 的类型为 `int`，所以 `d` 也是初值为 0 的 `int` 变量。
- E 是函数声明：声明一个名为 `e`、无参数、返回 `int` 的函数，并没有定义变量。这是著名的“声明歧义”入门例。

因此 A、B、C、D 全选，E 不选。依据：S7、S12

多选 4：答案 A、B、C、E

- **A 正确**。用限定名 `std::cout` 直接指出名字所属命名空间。
- **B 正确**。 using 指令可以出现在块作用域；它使指定命名空间中的名字可参与后续的不限定名称查找，所以裸写 `cout` 可找到 `std::cout`。
- **C 正确**。 using 声明只把特定名称 `std::cout` 带入当前作用域，随后的裸写 `cout` 合法；它比导入整个命名空间范围更窄。
- **D 错误**。题目排除了任何 using 机制，裸写 `cout` 无法在当前作用域找到。
- **E 正确**。 `std::cout` 和 `std::endl` 都使用了完整限定名；该语句输出 A、换行并刷新。

依据：S3、S4、S6

多选 5：答案 A、B、C、E

- **A 正确。** C/C++ 标准库的 `system` 把命令字符串交给宿主环境处理，但命令名 `pause` 及其效果由具体命令解释器决定。它不是标准 C++ 定义的跨平台暂停语句；此外规范调用应直接包含 `<cstdlib>` 并写 `std::system`。
- **B 正确。** `std::endl` 的定义效果包含“换行 + 刷新”。
- **C 正确。**仅要求输出换行时，`'\n'` 已表达所需字符，不强制每次刷新。
- **D 错误。**在 C++17 中，未初始化的自动存储期局部 `int` 具有不确定值；在通常的求值中读取它会导致未定义行为，绝不能假定为 0。
- **E 正确。**列表初始化会拒绝某些潜在信息丢失的窄化转换，如 `int x{3.9};`。

依据：S4、S7、S10

4.4 三、填空题详解

填空 1

三个空依次为 iostream、main、cout。完整程序是：

```cpp
#include <iostream>

int main()
{
    std::cout << "Hello, C++!" << '\n';
    return 0;
}
```

第一空构成标准头名；第二空给出标准入口函数名，大小写不能变；第三空与题目已经给出的 std:: 组合成标准输出流对象的限定名。多填尖括号、std:: 或括号都会破坏题目已有语法结构。依据：S1、S3、S5

填空 2

依次填写 //、/*、*/。前者从起始记号持续到下一换行之前；后者从 /* 持续到随后第一个 */，可跨行但不能嵌套。注意三者的斜杠方向与星号次序：块注释“斜杠星号开始，星号斜杠结束”。依据：S2

填空 3

依次填写 int、score、95。int 决定对象的整数类型；score 是该对象的名称；花括号中的 95 是初始化器提供的值。分号终止整个声明，不属于变量名或初值。依据：S7、S12

填空 4

答案为 3 2。因为 $17 = 5 \times 3 + 2$，故整数商 17 / 5 为 3，余数 17 % 5 为 2。中间的字符字面量' ' 精确输出一个空格，末尾没有换行要求。依据：S9

填空 5

第一空填“换行”或'\n'，第二空填 flush。更精确地说，std::endl 对流先执行与插入加宽后的换行字符等效的操作，再调用 flush()；它不是单纯的转义字符别名。依据：S4

4.5 四、综合题详解

综合 1：把手写程序修到可移植

评分点与问题定位（10 分）

原代码至少包含下列问题。前 7 项中的任意 6 项说明准确可得问题定位部分满分；第 8、9 项用于区分“勉强能在某环境运行”和“标准、可移植、表达清晰”。

序号	原写法/现象	问题与修正
1	`#Include <iostream>`	预处理指令名区分大小写，应写 `#include <iostream>`。
2	`Using namespace std`	关键字区分大小写，应为小写 `using`。
3	using 指令行末无分号	完整语法为 `using namespace std;`。
4	`int Main()`	C++ 区分大小写，程序入口必须名为全小写 `main`。
5	`int a = 10`	声明语句末尾缺少分号。
6	输出语句行末无分号	`cout << ... << endl;` 是完整表达式语句，末尾需要分号。
7	裸写 `cout`、`endl`	在原程序中 using 行本身不合法，名称无法按预期引入；最直接稳妥的修正是写 `std::cout`、`std::endl`。
8	`system` 未见声明来源	若确有必要调用标准库函数，应直接包含 `<cstdlib>` 并写 `std::system`，不能依赖其他头的偶然间接包含。
9	`system("pause")`	字符串被交给宿主命令解释器；`pause` 的存在和行为不是标准 C++ 保证。为得到可移植程序，应删除。普通命令行环境也不需要程序自行“按键暂停”。
10	无条件使用 `endl`	不是语法错误，但它会额外刷新流；只要求换行时用 `'\n'` 更准确地表达意图。

标准、简洁的参考程序

```cpp
#include <iostream>

int main()
{
    int a{10};
    std::cout << "a = " << a << '\n';
    return 0;
}
```

逐行说明

1. `#include <iostream>` 使 `std::cout` 的声明可用。
2. `int main()` 定义标准入口函数。
3. `int a{10};` 定义并列表初始化整数变量 `a`。
4. 输出表达式先写入字符串`"a = "`，再写入变量当前值 10，最后写入换行字符。
5. `return 0;` 向宿主环境报告正常结束；在 `main` 中可省略，但保留有助于初学者明确返回语义。

精确输出为：

```
a = 10
```

常见“修而未好”

只把 `Main` 改小写、补分号，却继续依赖 `system("pause")`，最多只能得到“在某些 Windows 命令解释器环境可能运行”的版本，不能满足题目要求的可移植性。把 `using namespace std;` 放在头文件或大型项目全局作用域也可能引入名称冲突；本例采用显式 `std::` 限定。

依据：S1、S3、S4、S5、S6、S10

综合 2：变量状态与块作用域追踪

第 1 问：精确输出（4 分）

```
3 1
4 8
7 8
```

每行最后都有`'\n'`，因此第三行之后也换行。

状态表推导

执行点	外层 a	b	q	r	说明
四个声明后	7	2	3	1	$7/2=3$，$7 \bmod 2=1$
第一次输出后	7	2	3	1	输出 `3 1`，状态不变
进入内层、声明内层 a	7（被遮蔽）	2	3	1	内层 a=q+r=4
执行 `b = a * 2` 后	7（被遮蔽）	8	3	1	当前可见的 a 是内层值 4；没有声明内层 b，故修改外层 b
第二次输出后	7（被遮蔽）	8	3	1	输出内层 a=4 与 b=8
离开内层块	7	8	3	1	内层 a 生存期结束，外层 a 再次可见
第三次输出后	7	8	3	1	输出 `7 8`

第 2 问：同名变量关系（4 分）

内外两个 a 来自两次不同声明，是两个独立对象。内层块中，较近的声明使内层 a 遮蔽外层 a；这不是覆盖、赋值或改名。离开内层块后，内层对象被销毁，外层 a 从始至终未被修改，所以仍为 7。

b 不同：内层没有重新声明 b，所以 b = a * 2; 找到并修改外层 b。离开内层块不会撤销这一赋值，因此最终 b=8。

第 3 问：消除遮蔽（2 分）

```
{
    int innerA{q + r};
    b = innerA * 2;
    std::cout << innerA << ' ' << b << '\n';
}
```

这段改写只改变局部变量名称，不改变初始化式、赋值目标和输出值，因此输出保持不变，同时避免读者误把内外两个 a 当成同一变量。

正确性与复杂度 所有语句执行固定次数，故时间复杂度为 $O(1)$；只使用固定数量的整数变量，额外空间复杂度为 $O(1)$。不存在除以 0：除数 b 在计算 q、r 时为 2。

高频错误

- 把 7/2 写成 3.5，忽略两个操作数均为整数。
- 认为内层 a 的值 4 会替换外层 a。
- 认为退出内层块会把 b “回滚”为 2；作用域结束不会撤销对外层对象已经完成的赋值。
- 漏掉输出中的空格或行末换行，导致“数值对、格式错”。

依据：S8、S9

综合 3：总秒数拆分

建模与数据类型（4 分）

一天有 $24 \times 60 \times 60 = 86400$ 秒。对剩余秒数依次做欧几里得除法：

$$d = S \,/\, 86400, \quad r_1 = S \bmod 86400,$$
$$h = r_1 \,/\, 3600, \quad r_2 = r_1 \bmod 3600,$$
$$m = r_2 \,/\, 60, \qquad s = r_2 \bmod 60.$$

输入上界是 10^{12}。常见 32 位 int 的最大值约为 2.147×10^9，无法容纳该上界；标准保证 long long 至少具有足够覆盖本题范围的宽度，故输入、中间余数与结果统一使用 long long。

参考程序（6 分）

```cpp
#include <iostream>

int main()
{
    long long totalSeconds{};
    std::cin >> totalSeconds;

    constexpr long long secondsPerDay{24LL * 60 * 60};
    constexpr long long secondsPerHour{60LL * 60};
    constexpr long long secondsPerMinute{60};

    long long remaining{totalSeconds};
    long long days{remaining / secondsPerDay};
    remaining %= secondsPerDay;

    long long hours{remaining / secondsPerHour};
    remaining %= secondsPerHour;

    long long minutes{remaining / secondsPerMinute};
    long long seconds{remaining % secondsPerMinute};

    std::cout << days << ' ' << hours << ' '
              << minutes << ' ' << seconds << '\n';
    return 0;
}
```

样例逐步计算

步骤	商（当前单位）	余数
$90061 \div 86400$	$d = 1$	3661
$3661 \div 3600$	$h = 1$	61
$61 \div 60$	$m = 1$	$s = 1$

所以输出 `1 1 1 1`。

正确性证明

由整数商余性质，第一步存在唯一的 d, r_1 使 $S = 86400d + r_1$ 且 $0 \le r_1 < 86400$。同理，$r_1 = 3600h + r_2$ 且 $0 \le r_2 < 3600$，故 $0 \le h < 24$；再有 $r_2 = 60m + s$ 且 $0 \le s < 60$，故 $0 \le m < 60$。代回可得

$$S = 86400d + 3600h + 60m + s,$$

且各较小单位都落在题目要求范围，所以算法输出的分解合法。商余分解在这些范围下唯一，故输出也是唯一正确答案。

复杂度（2 分） 除法、取余、赋值与输出的次数均与 S 大小无关，所以时间复杂度为 $O(1)$；变量数量固定，所以额外空间复杂度为 $O(1)$。

等价的直接写法 若希望保留原输入不修改，可直接写：

```
long long days    = totalSeconds / 86400;
long long hours   = totalSeconds % 86400 / 3600;
long long minutes = totalSeconds % 3600 / 60;
long long seconds = totalSeconds % 60;
```

运算符 `*`、`/`、`%` 同级并从左向右结合；这里额外写清单位常量通常更易维护。

输入 S	期望输出	覆盖目的
0	`0 0 0 0`	最小值
59	`0 0 0 59`	分钟边界之前
60	`0 0 1 0`	恰好一分钟
3599	`0 0 59 59`	小时边界之前
3600	`0 1 0 0`	恰好一小时
86399	`0 23 59 59`	天边界之前
86400	`1 0 0 0`	恰好一天
90061	`1 1 1 1`	四种单位均非零
10^{12}	`11574074 1 46 40`	最大值与 64 位数据类型

高频错误

- 用 `int` 保存 10^{12}，在常见平台发生范围溢出。
- 算完天数后仍直接用原数除以 3600，导致小时包含了整天对应的小时数。
- 把 `/` 与`%` 的作用互换：前者取商，后者取余。
- 常量写错：一天是 86400 秒，一小时是 3600 秒。
- 输出单位文字或逗号，而题目要求四个空格分隔的整数。

依据：S3、S9

综合 4：课程总评与格式化输出

公式翻译

题目已经给出数学模型。代码只需保证权重参与**浮点运算**：

$$0.20p + 0.30m + 0.50f.$$

三个输入虽是整数，但乘上浮点字面量后会按浮点规则计算，结果保存到 `double`。输出端再固定为两位小数。

参考程序（7 分）

```cpp
#include <iomanip>
#include <iostream>

int main()
{
    int usual{};
    int midterm{};
    int finalExam{};
    std::cin >> usual >> midterm >> finalExam;

    double finalScore{
        0.20 * usual + 0.30 * midterm + 0.50 * finalExam
    };

    std::cout << std::fixed << std::setprecision(2)
              << finalScore << '\n';
    return 0;
}
```

样例推导

$$0.20 \times 85 + 0.30 \times 90 + 0.50 \times 92 = 17 + 27 + 46 = 90.$$

固定两位小数后输出 `90.00`。

第 2 问：为什么不能写整数权重（3 分）

若写 `20 / 100`，两个字面量的类型都是 `int`，所以先做整数除法。由于 $20 < 100$，小数部分被舍弃，结果为 0；同理 `30 / 100` 和 `50 / 100` 也都为 0。于是错误表达式

```cpp
(20 / 100) * usual + (30 / 100) * midterm
                   + (50 / 100) * finalExam
```

会得到 0，而不是加权分数。

可行写法包括直接使用 `0.20`、`0.30`、`0.50`，或写 `20.0 / 100.0` 等，让至少一个操作数为浮点类型。

第 3 问：输出格式与复杂度（3 分）

`std::fixed` 选择定点表示；在该模式下，`std::setprecision(2)` 把精度解释为**小数点后 2 位**。如果没有 `std::fixed`，precision 通常控制有效数字个数，语义不同。流会按设定精度进行舍入并补足末尾 0，因此整数值 90 显示为 `90.00`。

程序只做固定次数的输入、乘法、加法与输出，时间复杂度为 $O(1)$；只使用固定数量变量，额外空间复杂度为 $O(1)$。

正确性说明

程序分别将三项输入乘以题目规定的权重 0.20、0.30、0.50，再相加；这与给定数学定义逐项一致。权重之和为 $0.20+0.30+0.50=1$，因此当三个成绩都在 $[0,100]$ 时，加权结果也在 $[0,100]$。最后的格式操纵只改变文本表示，不改变已经计算出的数学量，所以程序输出的两位小数表示满足题意。

避免二进制浮点的等价方案 本题权重都是整数百分比，可把“分数的百分之一”完全放在整数中计算：

```cpp
#include <iomanip>
#include <iostream>

int main()
{
    int usual{}, midterm{}, finalExam{};
    std::cin >> usual >> midterm >> finalExam;

    int hundredths{20 * usual + 30 * midterm + 50 * finalExam};
    std::cout << hundredths / 100 << '.'
              << std::setw(2) << std::setfill('0')
              << hundredths % 100 << '\n';
}
```

例如样例得到 `hundredths`=9000，整数部分为 90，余下两位为 00。这不是本题必需，但展示了“先缩放为整数、最后格式化”的精确计算思路。

输入	期望输出	覆盖目的
0 0 0	0.00	下界
100 100 100	100.00	上界与权重和为 1
100 0 0	20.00	单独验证 20% 权重
0 100 0	30.00	单独验证 30% 权重
0 0 100	50.00	单独验证 50% 权重
73 84 95	87.30	非整数结果与末尾补 0
85 90 92	90.00	题目样例

高频错误

- 使用整数权重 `20 / 100`，导致三项权重均先变为 0。
- 忘记包含 `<iomanip>`，却使用 `std::setprecision`。
- 只写 `setprecision(2)` 而不写 `fixed`，把“两位小数”误成“两位有效数字”。
- 权重错位、误写成简单平均，或用字符串拼接伪造小数位。

依据：S3、S9、S13

5 来源、检索说明与题目映射

检索与改编说明

本册在 2026 年 7 月 24–25 日进行了网页检索。技术事实优先依据公开的 C++ 工作草案条款，并用 GCC、Microsoft 官方文档补充实现层面的头文件搜索和宿主命令说明。题目均围绕这些规则**重新设计、组合和改编**，不是对网络题库的逐字复制。虽然工作草案页面会随标准演进更新，本册只采用在 C++17 中已经成立且与入门主题相关的稳定规则；涉及未初始化局部整数时，解析明确以 C++17 语义为准。

5.1 权威资料

编号	资料	本册采用的事实
S1	C++ draft: main function	程序的全局 `main`、返回类型 `int`、允许的参数形式，以及到达末尾等价于返回 0。
S2	C++ draft: Comments	`//` 与 `/*...*/` 的终止规则、块注释不嵌套、注释记号在另一注释内部的处理。
S3	C++ draft: Standard iostream objects	`<iostream>` 所声明的标准流对象，以及 `std::cout` 与标准输出的关联。
S4	C++ draft: Output-stream manipulators	`std::endl` 插入换行字符后调用 `flush()` 的精确效果。
S5	C++ draft: Source file inclusion	`#include` 的源文件包含形式、头搜索由实现规定，以及尖括号/引号形式的可移植用途。
S6	C++ draft: using-directive	`using namespace` 的语法、可出现于命名空间或块作用域，以及对不限定名称查找的影响。
S7	C++ draft: List-initialization	花括号列表初始化及其对浮点到整数等窄化转换的拒绝。
S8	C++ draft: Scope	复合语句引入块作用域，块变量与外层/内层名称查找。
S9	C++ draft: Multiplicative operators	整数除法舍弃小数部分、`/` 与`%` 的商余关系、除数为 0 的边界。
S10	Microsoft Learn: system, _wsystem	`system` 把命令传递给命令解释器，命令执行依赖宿主环境。
S11	GCC: Include Syntax	尖括号头与引号头的典型搜索顺序，以及包含发生在预处理阶段。
S12	Microsoft Learn: Declarations and definitions	实体先声明后使用、局部变量、`std` 命名空间和常见变量定义示例。
S13	C++ draft: Header <iomanip>; standard manipulators	`setprecision`、`setw`、`setfill` 等格式操纵符的声明与流状态效果。

5.2 题目–来源映射

资料	对应题目	核对重点
S1	单选 1；多选 1；填空 1；综合 1	`main` 形式、返回与大小写
S2	单选 5；多选 2；填空 2	行注释、块注释、不可嵌套
S3	单选 1–3；多选 4；填空 1；综合 1、3、4	标准流对象和输出链
S4	单选 4；多选 4、5；填空 5；综合 1	换行与刷新区别
S5、S11	单选 2；填空 1；综合 1	头文件包含与搜索
S6	单选 1；多选 4；综合 1	using 指令与限定名
S7、S12	单选 6、7、10；多选 3、5；填空 3；综合 1、4	声明、初始化、窄化和名称
S8	单选 8；综合 2	块作用域与遮蔽
S9	单选 9；填空 4；综合 2–4	整数商余、表达式建模
S10	多选 5；综合 1	`system("pause")` 的环境依赖
S13	综合 4	两位小数和流格式状态

5.3 最终复核清单

检查项	结果	证据
题量	通过	10 单选、5 多选、5 填空、4 综合题，共 24 题。
答案位置	通过	第 3 节仅含题目，第 4 节集中给出答案。
答案唯一性	通过	单选每题唯一；多选题干明确“正确的有”，答案集合逐项核对。
难度均衡	通过	9 基础、12 中等、3 提高；综合题由纠错、读码到建模与精度递进。
标准基线	通过	除故意设置的错误代码外，参考程序按标准 C++17 编写，不依赖 IDE 扩展。
解析深度	通过	客观题逐项辨析；综合题含建模、代码、推导/证明、复杂度、误区与边界测试。
来源可追溯	通过	文末列出 13 组权威网页资料，并建立题目–来源映射。

——完——

建议二次练习时遮住答案区，并为每个错误选项写一句反证理由。